MOTOR DRIVER & PWM

LAB 4

Group Members: Dipen Gaihre and Lotus Nyaupane

Course: Controllers and Electronics

Date: 13.11.2025

Contents

1. Objectives 1
2. Theory 1
3. Equipment 2
4. Wiring 2
5. Tasks and Methods 4
   A. Basic PWM Motor Control (Task 1) 4
   B. Drive Motor Forward and Backward (Task 2) 5
   C. Accelerate Motor From 0% to 100% (task 3) 6
   D. Button Control (Task 4) 7
6. Results and Observations 10
7. Conclusion 10
8. References 11

1. Objectives

The purpose of this lab was to learn how to control a DC motor using the Raspberry Pi and a Cytron MD13S motor driver. We used PWM to change the motor speed and a direction pin to change the rotation direction. Later, we added buttons to control the motor manually.

This lab helped us understand how PWM, H-bridges, and motor drivers work together.

2. Theory

PWM (Pulse Width Modulation)

PWM is a way to control how much power a motor gets.
It switches the signal ON and OFF very fast.

- Higher duty cycle → more power → motor spins faster
- Lower duty cycle → less power → motor spins slower

For example:

- 50% duty = ON half the time
- 100% duty = full power
- 0% duty = off

H-Bridge

An H-bridge is a circuit that lets current flow in two directions.
This allows the motor to spin forward or backward.

The motor driver (MD13S) already has this built in, so we just control:

- Direction Pin → chooses rotation direction
- PWM Pin → chooses motor speed

3. Equipment

Equipment's we used in lab are:

- Raspberry Pi Zero 2W
- Cytron MD13S Motor Driver
- 12V DC Motor
- External 12V Power Supply
- Grove Dual Button
- Breadboard
- Jumper Wires
- 40-pin connector
- Python with pigpio library

4. Wiring

We connected the Raspberry Pi to the Cytron MD13S motor controller:

- One GPIO pin for Direction
- One GPIO pin for PWM speed control
- 12V power supply only to the motor driver
- The motor connected to the output terminals of MD13S
- Buttons connected using the Grove Base Hat (GPIO 16 & 17)

We have some pictures of wiring from the Lab:

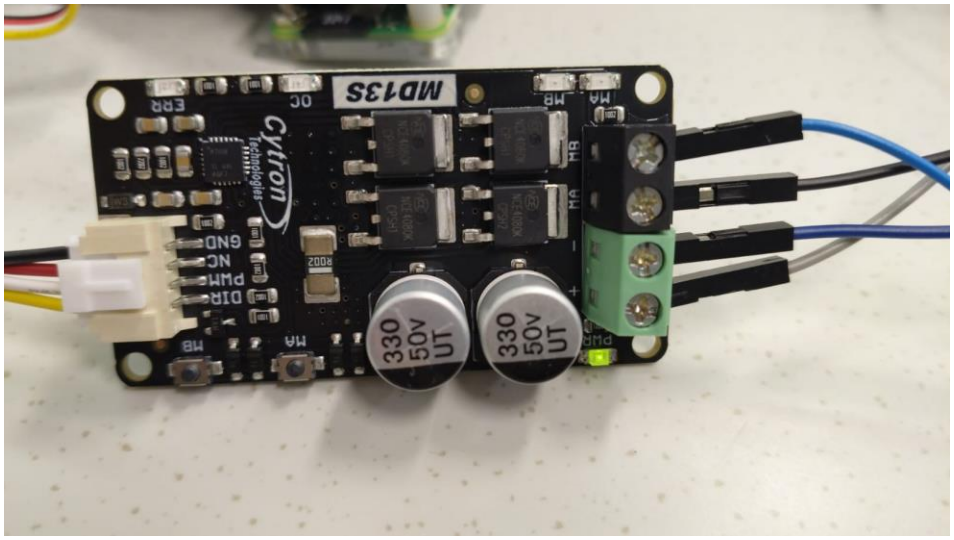

Fig: picture of motor controller wiring

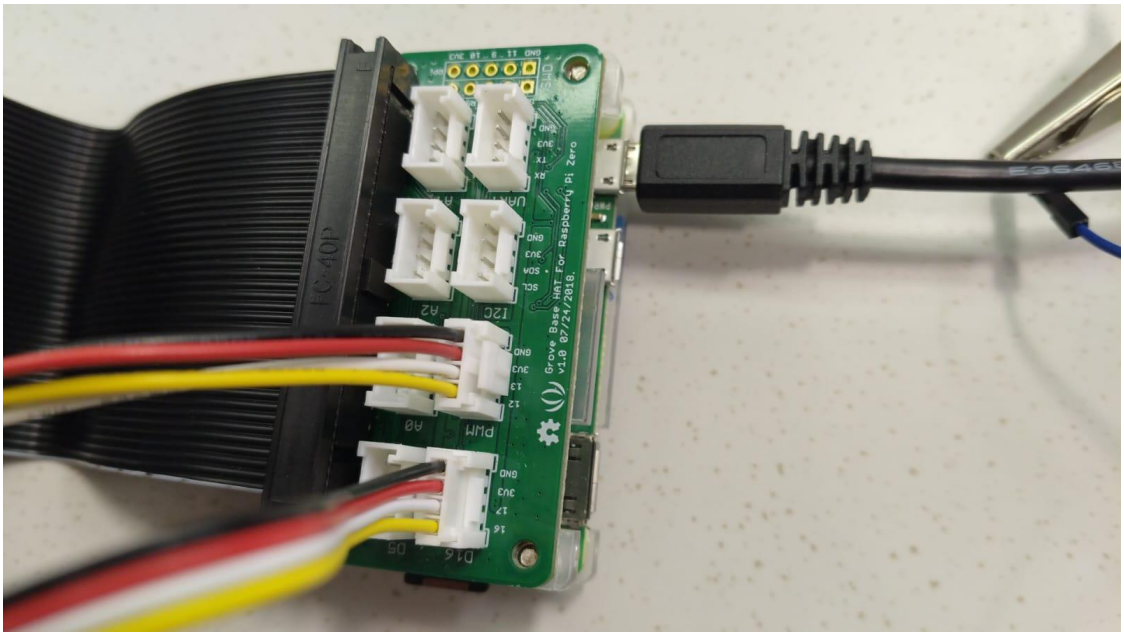

***Fig: picture of Pi connection with motor controller and* Grove Dual Button**

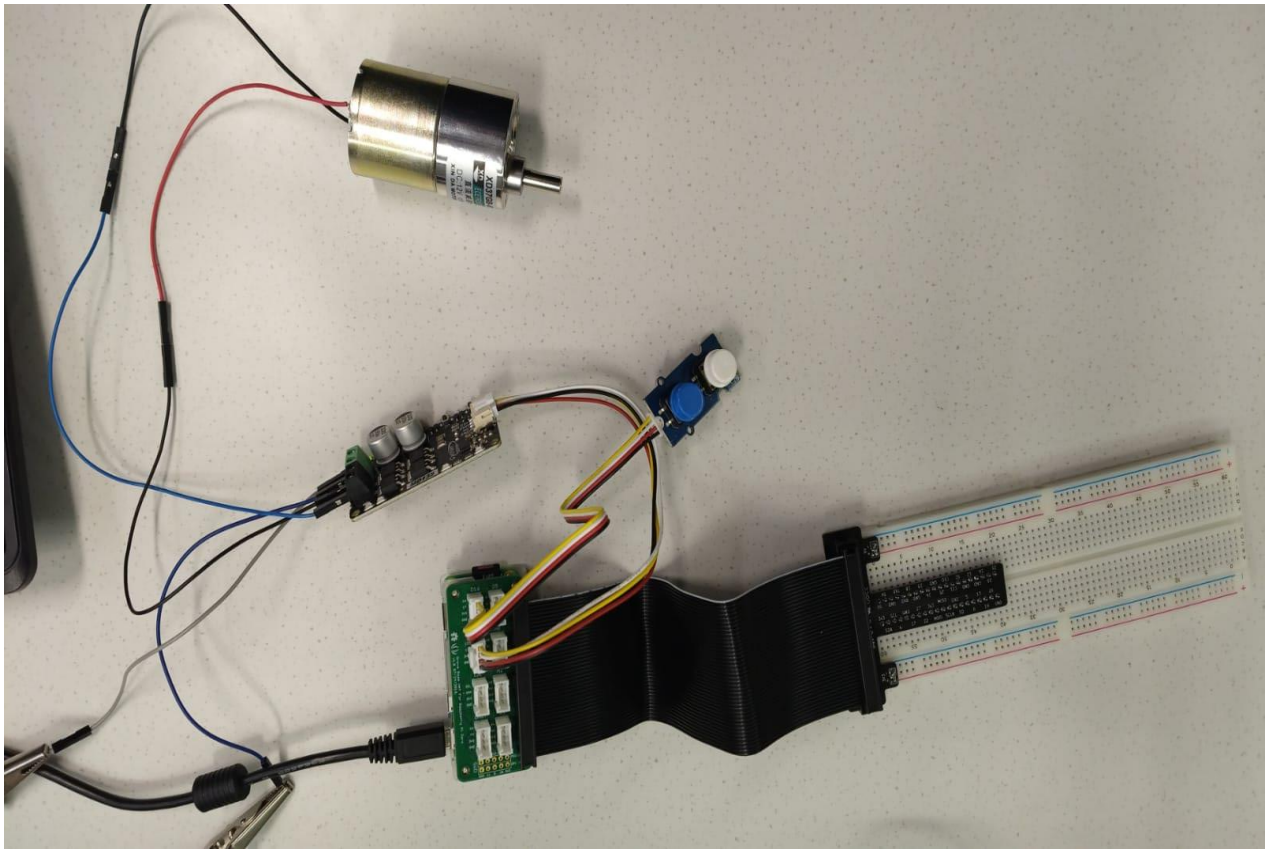

fig: picture of full wiring setup

5. Tasks and Methods

A. Basic PWM Motor Control (Task 1)

In this task, we learned how to use pigpio to send PWM signals to the motor driver. We used a hardware-PWM-enabled pin so the motor speed would be smooth.

We tried different duty cycles and confirmed the motor speed changed.

Code:

```
labuser@RiiA103RPiZe29: ~/lc
  GNU nano 8.4                                        lab4
import pigpio
import time

PWM_PIN = 13      # Hardware PWM pin
DIR_PIN = 12
      # Motor direction pin
PWM_FREQ = 10000  # 10 kHz PWM signal

pi = pigpio.pi()
if not pi.connected:
    exit()

# Setup pins
pi.set_mode(DIR_PIN, pigpio.OUTPUT)

# Forward direction
pi.write(DIR_PIN, 1)

# Set PWM (Duty range 0-1,000,000)
pi.hardware_PWM(PWM_PIN, PWM_FREQ, 500000)  # 50% duty

time.sleep(3)

# Stop motor
pi.hardware_PWM(PWM_PIN, PWM_FREQ, 0)

pi.stop()
```

Fig: Task 1 code screenshot

B. Drive Motor Forward and Backward (Task 2)

We used:

- 100% duty cycle for full speed forward
- 50% duty cycle for medium speed backward

Changing the direction pin made the motor spin in the opposite direction.

```
labuser@RiiA103RPiZe29: ~/lc
GNU nano 8.4                                    lab4_2
import pigpio
import time

PWM_PIN = 13
DIR_PIN = 12
PWM_FREQ = 10000

pi = pigpio.pi()
pi.set_mode(DIR_PIN, pigpio.OUTPUT)

# -------- FORWARD 100% --------
pi.write(DIR_PIN, 1)
pi.hardware_PWM(PWM_PIN, PWM_FREQ, 1000000)  # 100%
time.sleep(3)

# Stop before reversing
pi.hardware_PWM(PWM_PIN, PWM_FREQ, 0)
time.sleep(1)

# -------- BACKWARD 50% --------
pi.write(DIR_PIN, 0)
pi.hardware_PWM(PWM_PIN, PWM_FREQ, 500000)  # 50%
time.sleep(3)

# Stop
pi.hardware_PWM(PWM_PIN, PWM_FREQ, 0)
pi.stop()
```

Fig: Task 2 code screenshot

C. Accelerate Motor From 0% to 100% (task 3)

We made a loop that slowly increases the duty cycle from 0 to 100.
This created a smooth acceleration effect.

We also tested this in the terminal and confirmed the duty increased step-by-step.

```
labuser@RiiA103RPiZe29: ~/lc
  GNU nano 8.4                                        lab4_3
import pigpio
import time

PWM_PIN = 13
DIR_PIN = 12
PWM_FREQ = 10000

pi = pigpio.pi()
pi.set_mode(DIR_PIN, pigpio.OUTPUT)

pi.write(DIR_PIN, 1)  # Forward

for duty in range(0, 1000001, 50000):  # 0 → 100% in steps
    pi.hardware_PWM(PWM_PIN, PWM_FREQ, duty)
    print("Duty:", duty / 10000, "%")
    time.sleep(0.1)

# Stop
pi.hardware_PWM(PWM_PIN, PWM_FREQ, 0)
pi.stop()
```

Fig: Task 3 code screenshot

```
labuser@RiiA103RPiZe29: ~/lo
      self.sl, _PI_CMD_HP, gpio, PWMfreq, 4, extents))
  File "/usr/local/lib/python3.13/dist-packages/pigpio-1.78-py3.13.egg/pigpio.py", line 1011, in _u2i
    raise error(error_text(v))
pigpio.error: 'GPIO has no hardware PWM'
labuser@RiiA103RPiZe29:~/lotus_dipens $ nano lab4
labuser@RiiA103RPiZe29:~/lotus_dipens $ python3 lab4
labuser@RiiA103RPiZe29:~/lotus_dipens $ python3 lab4
labuser@RiiA103RPiZe29:~/lotus_dipens $ python3 lab4
labuser@RiiA103RPiZe29:~/lotus_dipens $ nano lab4
labuser@RiiA103RPiZe29:~/lotus_dipens $ python3 lab4
labuser@RiiA103RPiZe29:~/lotus_dipens $ python3 lab4
labuser@RiiA103RPiZe29:~/lotus_dipens $ python3 lab4
labuser@RiiA103RPiZe29:~/lotus_dipens $ nano lab4
labuser@RiiA103RPiZe29:~/lotus_dipens $ python3 lab4
labuser@RiiA103RPiZe29:~/lotus_dipens $ python3 lab4
labuser@RiiA103RPiZe29:~/lotus_dipens $ nano lab4
labuser@RiiA103RPiZe29:~/lotus_dipens $ y
-bash: y: command not found
labuser@RiiA103RPiZe29:~/lotus_dipens $ python3 lab
python3: can't open file '/home/labuser/lotus_dipens/lab': [Errno 2] No such file or directory
labuser@RiiA103RPiZe29:~/lotus_dipens $ python3 lab4
labuser@RiiA103RPiZe29:~/lotus_dipens $ nano lab4
labuser@RiiA103RPiZe29:~/lotus_dipens $ nano lab4
labuser@RiiA103RPiZe29:~/lotus_dipens $ nano lab4_2
labuser@RiiA103RPiZe29:~/lotus_dipens $ python3 lab4_2
labuser@RiiA103RPiZe29:~/lotus_dipens $ python3 lab4_2
labuser@RiiA103RPiZe29:~/lotus_dipens $ nano lab4_2
labuser@RiiA103RPiZe29:~/lotus_dipens $ nano lab4_3
labuser@RiiA103RPiZe29:~/lotus_dipens $ python3 lab4_3
Duty: 0.0 %
Duty: 5.0 %
Duty: 10.0 %
Duty: 15.0 %
Duty: 20.0 %
Duty: 25.0 %
Duty: 30.0 %
Duty: 35.0 %
Duty: 40.0 %
Duty: 45.0 %
Duty: 50.0 %
Duty: 55.0 %
Duty: 60.0 %
Duty: 65.0 %
Duty: 70.0 %
Duty: 75.0 %
Duty: 80.0 %
Duty: 85.0 %
Duty: 90.0 %
Duty: 95.0 %
Duty: 100.0 %
labuser@RiiA103RPiZe29:~/lotus_dipens $
```

fig: terminal screenshot of Task 3 running

D. Button Control (Task 4)

We connected the Grove Dual Button so that:

- One button changed the motor direction, but only when duty was low
- The other button increased the motor speed like a gas pedal

This made the motor easy to control manually.

labuser@RiiA103RPiZe29: ~/l

```
  GNU nano 8.4
import pigpio
import time

# Motor pins
PWM_PIN = 13       # Hardware PWM
DIR_PIN = 12       # Direction pin (free GPIO)

# Button pins
BTN_ACCEL = 16     # Button A
BTN_DIR = 17       # Button B

PWM_FREQ = 10000   # 10 kHz PWM

pi = pigpio.pi()
if not pi.connected:
    print("pigpiod not running!")
    exit()

# Setup motor pins
pi.set_mode(DIR_PIN, pigpio.OUTPUT)

# Setup buttons
pi.set_mode(BTN_ACCEL, pigpio.INPUT)
pi.set_mode(BTN_DIR, pigpio.INPUT)

# Enable pull-ups (buttons active LOW)
pi.set_pull_up_down(BTN_ACCEL, pigpio.PUD_UP)
pi.set_pull_up_down(BTN_DIR, pigpio.PUD_UP)

# Motor state
direction = 1
duty = 0

pi.write(DIR_PIN, direction)

print("Motor control running... Press buttons.")

try:
    while True:
        accel_pressed = pi.read(BTN_ACCEL) == 0  # Active low
        dir_pressed = pi.read(BTN_DIR) == 0

        # ---- ACCEL BUTTON (GPIO 16) ----
        if accel_pressed:
            duty = min(1000000, duty + 25000)  # Increase duty
        else:
            duty = max(0, duty - 15000)  # Natural slow-down

        # ---- DIRECTION BUTTON (GPIO 17) ----
        if dir_pressed and duty < 100000:  # Must be slow to switch direction
            direction = 1 - direction
            pi.write(DIR_PIN, direction)
            print("Direction changed:", direction)
            time.sleep(0.3)  # Debounce

        # Apply PWM
        pi.hardware_PWM(PWM_PIN, PWM_FREQ, duty)

        time.sleep(0.02)

except KeyboardInterrupt:
    print("\nStopping motor...")

finally:
    pi.hardware_PWM(PWM_PIN, PWM_FREQ, 0)
    pi.stop()
```

Fig: Task 4 code screenshot

```
labuser@RiiA103RPiZe29: ~/lc
Duty: 60.0 %
Duty: 65.0 %
Duty: 70.0 %
Duty: 75.0 %
Duty: 80.0 %
Duty: 85.0 %
Duty: 90.0 %
Duty: 95.0 %
Duty: 100.0 %
labuser@RiiA103RPiZe29:~/lotus_dipens $ nano lab4_3
labuser@RiiA103RPiZe29:~/lotus_dipens $ nano lab4_4
labuser@RiiA103RPiZe29:~/lotus_dipens $ python3 lab4_4
Motor control running... Press buttons.
Direction changed: 0
Direction changed: 1
Direction changed: 0
Direction changed: 1
Direction changed: 0
Direction changed: 1
Direction changed: 0
Direction changed: 1
Direction changed: 0
Direction changed: 1
Direction changed: 0
Direction changed: 1
Direction changed: 0
Direction changed: 1
Direction changed: 0
Direction changed: 1
Direction changed: 0
Direction changed: 1
Direction changed: 0
Direction changed: 1
Direction changed: 0
Direction changed: 1
Direction changed: 0
Direction changed: 1
Direction changed: 0
Direction changed: 1
Direction changed: 0
Direction changed: 1
Direction changed: 0
Direction changed: 1
Direction changed: 0
Direction changed: 1
Direction changed: 0
Direction changed: 1
Direction changed: 0
Direction changed: 1
Direction changed: 0
```

Fig: terminal screenshot of Task 4 running

6. Results and Observations

- PWM worked well for controlling the motor speed.
- Hardware PWM was smoother than software PWM.
- The direction pin correctly reversed the motor.
- The acceleration loop showed how duty cycle affects speed.
- Buttons worked, but we had to control direction only at low speed to avoid sudden force on the motor.

Everything behaved as expected.

7. Conclusion

In this lab, we learned how to control a motor using PWM, a direction pin, and a motor driver. We practiced changing speed, reversing direction, and using buttons to interact with the motor. The lab also helped us understand why PWM frequency matters and how H-bridges make motor control simple.

Overall, this was a good introduction to motor control using microcontrollers.
And working on pervious labs make very easier to complete the tasks of this lab because we understand how to use all equipment's on pervious labs.

8. References

We used classroom instructions, datasheets, help from classmates, help from teachers and AI tools like ChatGPT to complete this overall Lab tasks.

The End